CampusSphere

Campus Opportunity Hub

UML Behavioral, Implementation and Environmental Views

Rajwardhan Singh - 24CSB1A52

Ramitha Ramesh - 24CSB1A53

Rayi Charan Harsha - 24CSB1A54

Version 1.0

Date: 02-Nov-2026

1. Introduction

This document presents the dynamic and physical UML models of CampusSphere - Campus Opportunity Hub. It continues the structural and use-case models (Class, Object and Use Case diagrams) already prepared for the project and covers the remaining views described in the Rational Rose tutorial: the Behavioral View (Sequence, Collaboration, State-Chart and Activity diagrams), the Implementation View (Component diagram) and the Environmental View (Deployment diagram).

View	Diagram	Question it answers	Figure
Behavioral	Sequence	In what time order do objects exchange messages to complete a use case?	Figure 1
Behavioral	Collaboration	Which objects are linked, and how do they cooperate?	Figure 2
Behavioral	State-Chart	Which states does an Opportunity / Application pass through during its life?	Figures 3 and 4
Behavioral	Activity	What is the workflow, and who performs each step?	Figure 5
Implementation	Component	Which software components exist and how do they depend on each other?	Figure 6
Environmental	Deployment	On which nodes does the software run and how are the nodes connected?	Figure 7

The scenario modelled in the behavioral diagrams is Use Case 3 (Apply for an Opportunity and Track Application Status) from the CampusSphere Use Cases Document v1.2, because it touches every major part of the system: the student, the web client, the API, authentication, the opportunity and application records, the notification service and the external e-mail/SMS gateway. The state-chart diagrams model the two entities with a real life cycle, the Opportunity and the Application.

Technology choices shown in the component and deployment diagrams (single-page web client, REST/JSON API, JWT-based authentication, a relational database, Docker containers started with Docker Compose) follow the non-functional requirements of the SRS. Product-specific details such as the database brand or the hosting provider are deliberately left generic and can be replaced with the team's actual choices.

2. Behavioral View

2.1 Sequence Diagram

A sequence diagram shows object interactions arranged in time order and is normally drawn for one use case. Figure 1 realizes Use Case 3 in three phases: browsing the opportunity, submitting the application, and tracking its status.

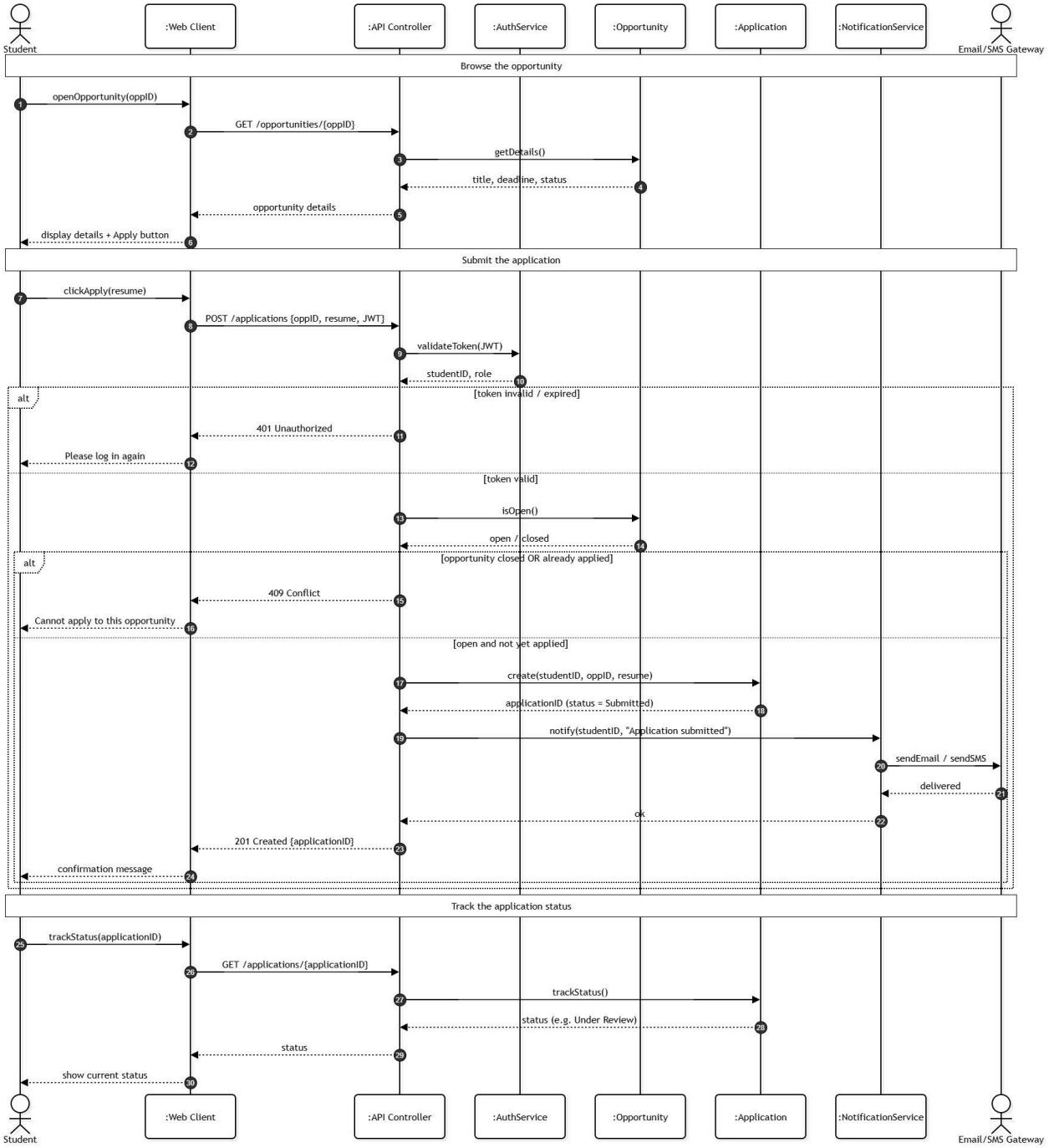


Figure 1: Sequence Diagram - Apply for an Opportunity and Track Application Status

The participants, from left to right, are the Student (actor), the Web Client, the API Controller, the AuthService, the Opportunity and Application objects, the NotificationService and the external Email/SMS Gateway (actor). In the first phase the student opens an opportunity; the web client requests it from the API, the controller asks the Opportunity object for its details and the data is

returned for display together with the Apply button. In the second phase the student clicks Apply and the client posts the application together with the JWT. The controller first asks AuthService to validate the token. If the token is invalid or expired the flow ends with a 401 response and a prompt to log in again. Otherwise the controller asks the Opportunity whether it is still open; if the opportunity is closed or the student has already applied, a 409 Conflict is returned. Only when both checks pass does the controller create the Application (status Submitted), ask the NotificationService to notify the student, and return 201 Created, while the NotificationService calls the e-mail/SMS gateway to deliver the confirmation. In the third phase the student later asks for the status and the controller reads it from the Application object. The nested alt fragments express the two guard conditions (valid token; open and not yet applied), numbered messages 1 to 30 give the exact order, and the narrow rectangles on the lifelines are activation bars showing when each object is busy.

2.2 Collaboration Diagram

A collaboration (communication) diagram shows the same interaction as the sequence diagram but emphasizes the links between objects instead of the time axis. Messages are ordered with a hierarchical sequence number, as in the Rational Rose example.

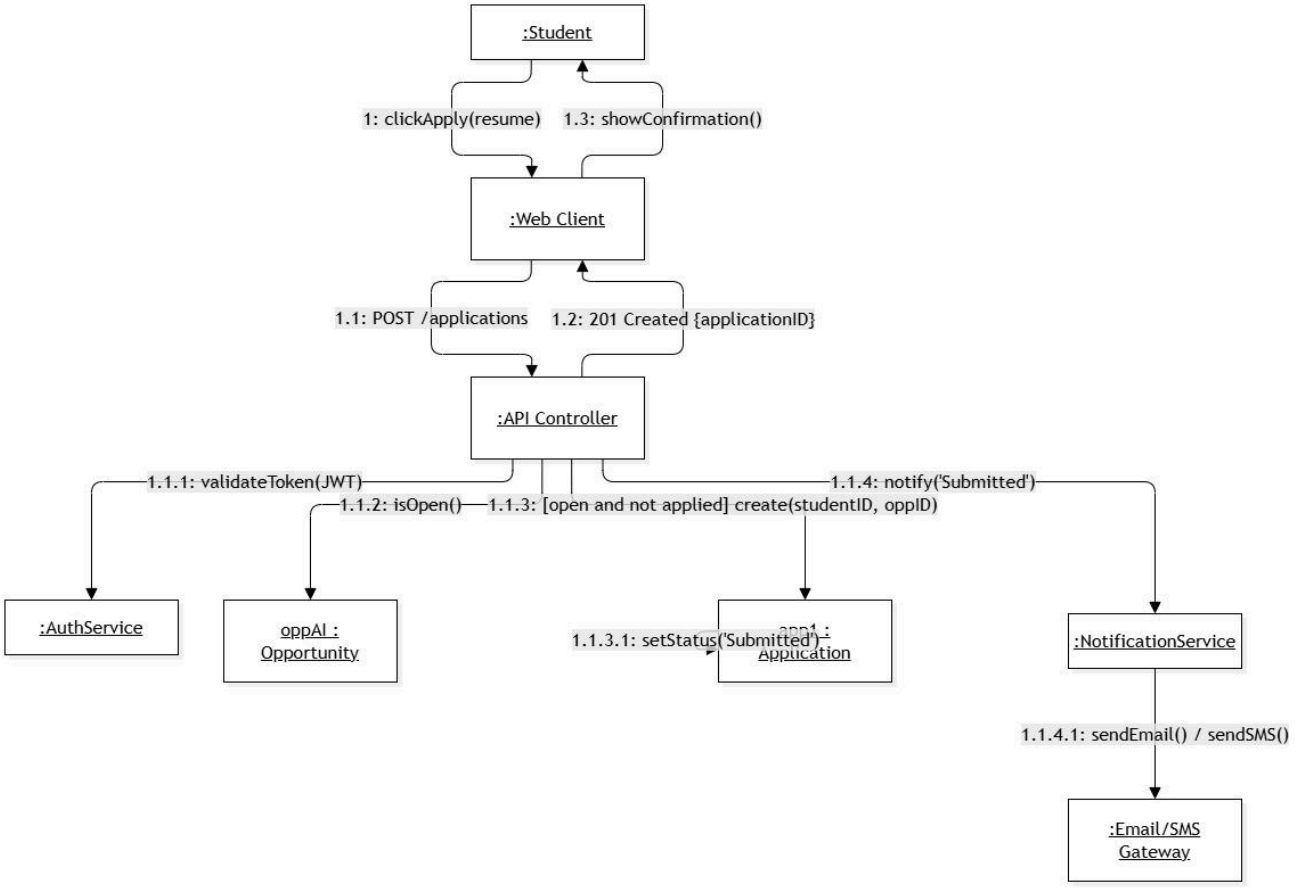


Figure 2: Collaboration Diagram - Apply for an Opportunity

Each box is an object (instance : Class) and each line is a link along which messages travel. Message 1 (clickApply) goes from the Student to the Web Client, which sends message 1.1 (POST /applications) to the API Controller. The numbering 1.1.1 to 1.1.4 shows that the controller then performs four nested calls in order: validateToken on the AuthService, isOpen on the Opportunity, create on the Application (guarded by the condition that the opportunity is open and not already applied) and notify on the NotificationService, which in turn sends message 1.1.4.1 to the Email/SMS Gateway. Message 1.1.3.1, setStatus, is a self-delegation: the Application object sets its own status to Submitted. Finally the controller answers the client with 1.2 (201 Created) and the client shows the confirmation to the student as message 1.3. Compared with Figure 1, this view makes it easy to see that the API Controller is the central coordinator with five collaborators, which is useful when judging coupling, whereas the sequence diagram is better for reading the order of events.

2.3 State-Chart Diagrams

State-chart diagrams model the discrete stages of one object's lifetime, the events that cause transitions, and the actions performed on entering, staying in and leaving a state. Two entities of CampusSphere have a meaningful life cycle.

2.3.1 Opportunity life cycle

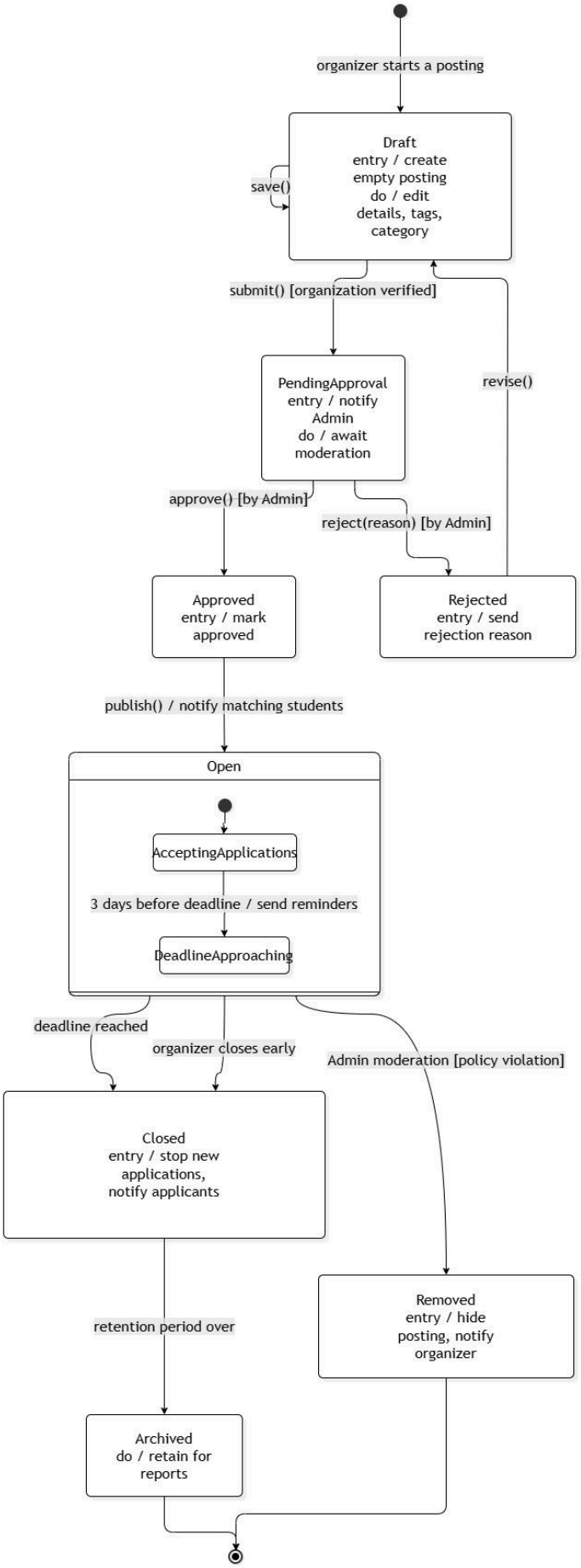


Figure 3: State-Chart Diagram - Lifecycle of an Opportunity

An Opportunity starts in the Draft state when an organizer begins a posting; while drafting, save() returns it to the same state. The organizer calls submit(), which is allowed only if the organization account is verified (guard condition), and the posting moves to PendingApproval, where the Admin is notified. The Admin either approves it, moving it to Approved, or rejects it with a reason, moving it to Rejected; the organizer can revise() a rejected posting, which returns it to Draft. After approval, publish() moves the posting into the composite state Open and notifies matching students. Open contains two sub-states: AcceptingApplications, where each applicationReceived event is a self-transition, and DeadlineApproaching, entered three days before the deadline, whose entry action sends reminders to students who bookmarked the posting. An Open posting leaves that state in one of three ways: the deadline is reached or the organizer closes it early, both leading to Closed (new applications are stopped and applicants notified), or an Admin removes it for a policy violation, leading to Removed (the posting is hidden and the organizer notified). A Closed posting is eventually moved to Archived after the retention period, and both Archived and Removed are final states.

State	Entry / do action	Leaves on
Draft	Create empty posting; edit details, tags, category	submit() [organization verified]
PendingApproval	Notify Admin; await moderation	approve() / reject(reason)
Rejected	Send rejection reason to organizer	revise()
Approved	Mark approved	publish()
Open	Accept applications; send deadline reminders	deadline reached; early close; moderation removal
Closed	Stop new applications; notify applicants	retention period over
Removed	Hide posting; notify organizer	final state
Archived	Retain data for reports	final state

2.3.2 Application life cycle

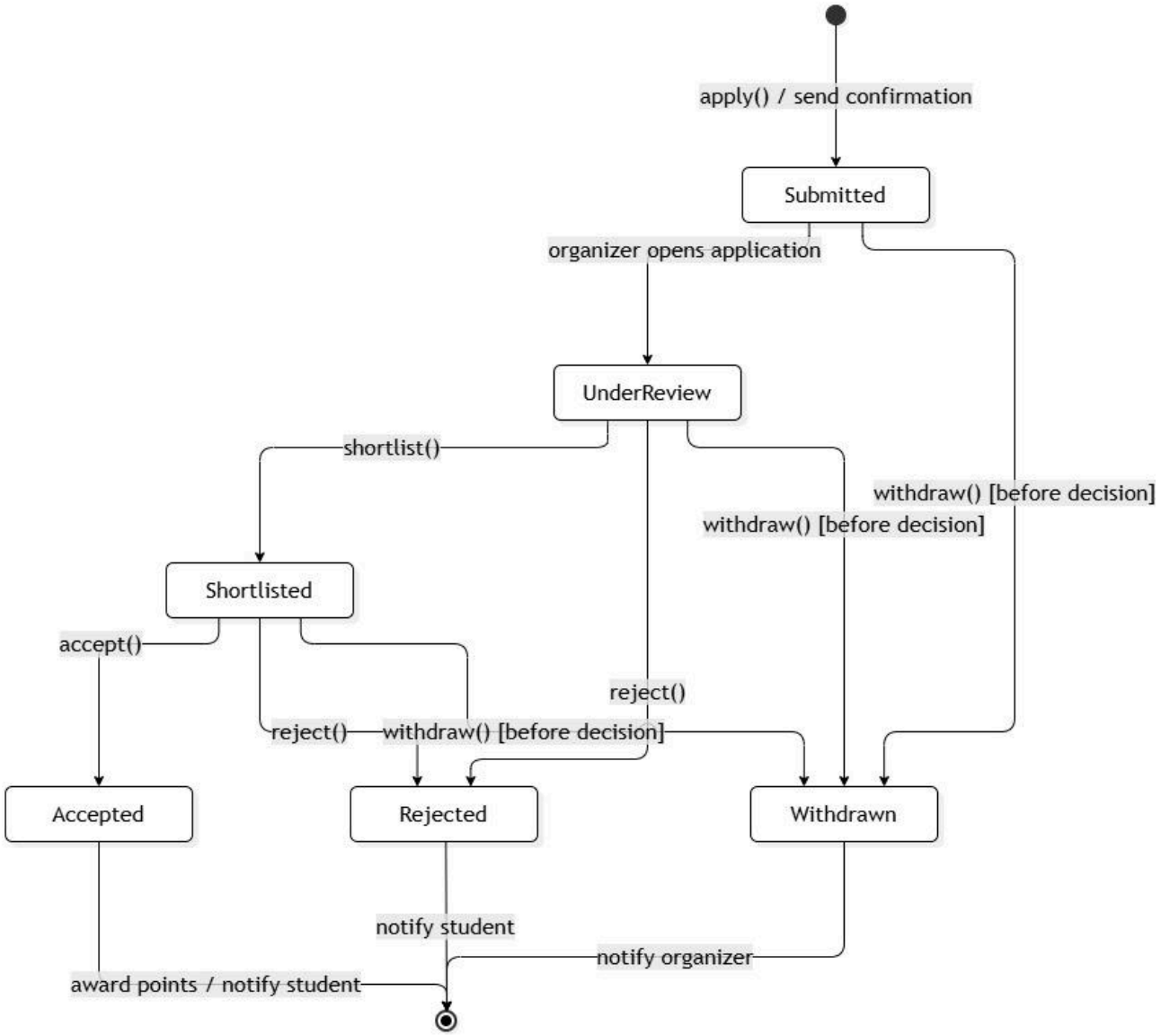


Figure 4: State-Chart Diagram - Lifecycle of an Application

An Application is created in the Submitted state when the student applies, and a confirmation is sent. When the organizer opens it, it becomes UnderReview; from there the organizer can shortlist it (Shortlisted) or reject it (Rejected). A Shortlisted application is finally accepted (Accepted, which awards points to the student) or rejected. At any point before a decision the student may withdraw it, moving it to Withdrawn and notifying the organizer. Accepted, Rejected and Withdrawn are final states, and each entry action in the diagram corresponds to the status string that the student sees when tracking the application in Use Case 3.

2.4 Activity Diagram

An activity diagram models the workflow of a process and, with swimlanes, shows which participant performs each activity. Figure 5 follows the whole journey from login to the organizer's decision.

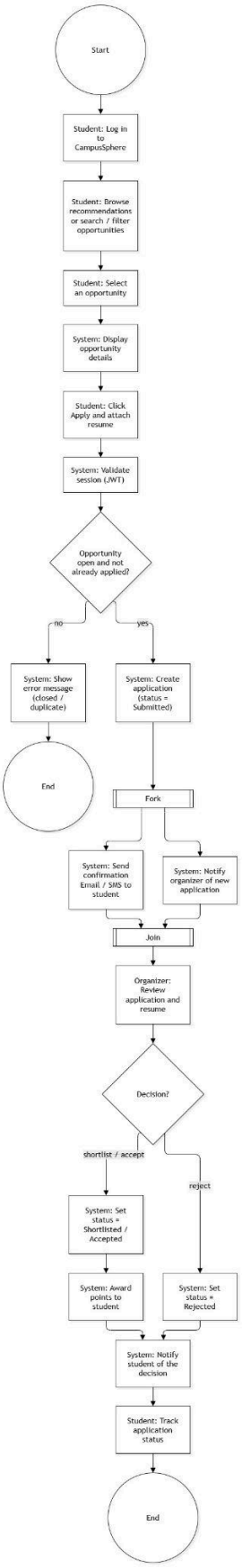


Figure 5: Activity Diagram - Discover, Apply and Review an Application

The diagram has three swimlanes: Student, CampusSphere System and Organizer. The student logs in, browses recommendations or searches and filters, selects an opportunity and, after the system displays its details, clicks Apply and attaches a resume. The system validates the session and reaches a decision node: if the opportunity is not open or the student has already applied, an error message is shown and the flow ends; otherwise an application with status Submitted is created. At this point a fork splits the flow into two parallel activities, sending the confirmation e-mail/SMS to the student and notifying the organizer, and a join waits for both before continuing. The organizer reviews the application and resume and a second decision node routes the flow: shortlist or accept sets the status to Shortlisted/Accepted and awards points to the student, while reject sets the status to Rejected. The two branches merge, the system notifies the student of the decision, and the student can track the final status. The activity diagram differs from the state-chart of Figure 4 in that it describes the process across several actors, whereas the state-chart describes the discrete stages of one object.

3. Implementation View

3.1 Component Diagram

A component diagram gives a physical view of the software: its components, their organization into layers and the dependencies among them. Figure 6 shows the implementation structure of CampusSphere.

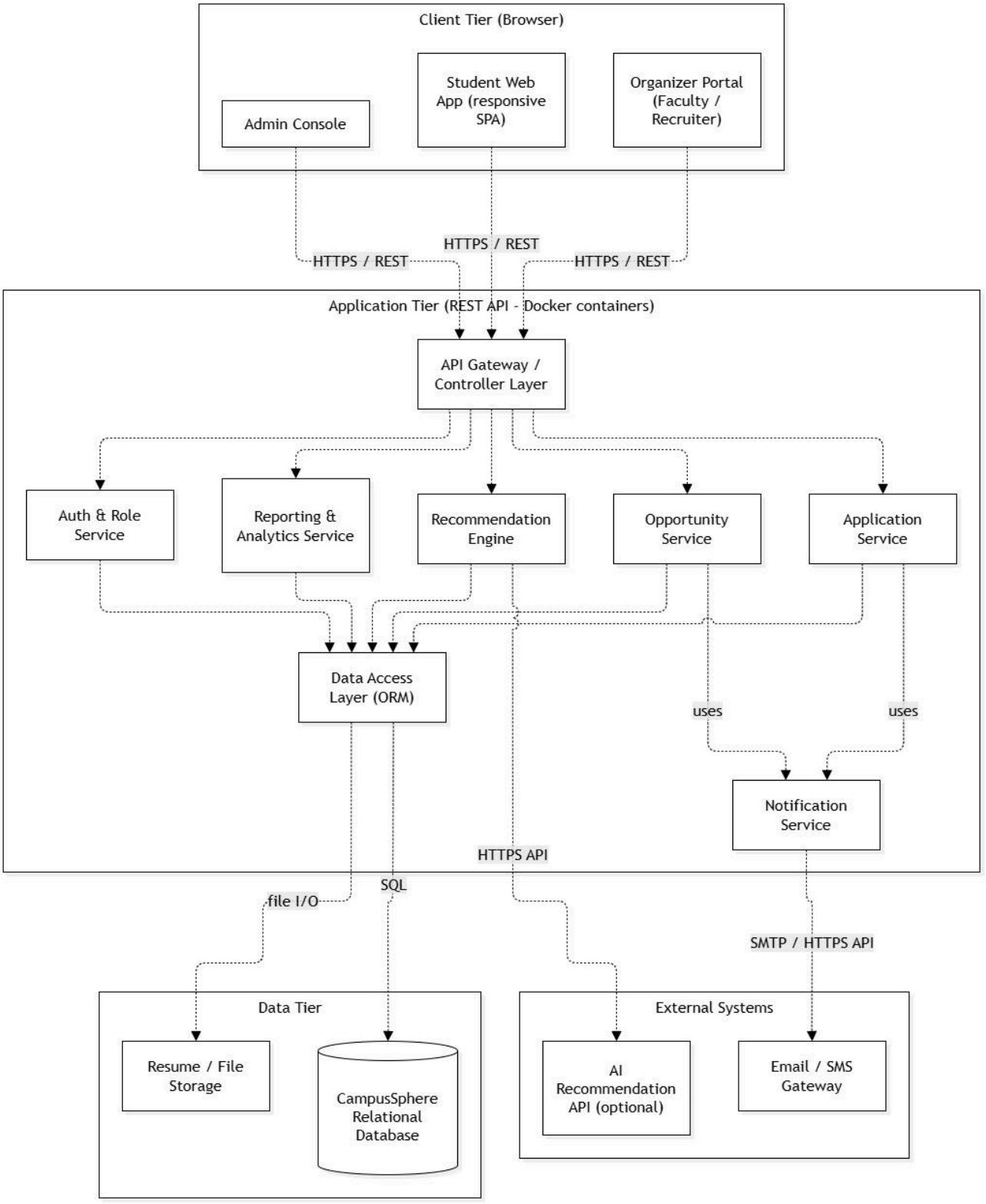


Figure 6: Component Diagram - CampusSphere (Implementation View)

The system is organized into four packages. The Client Tier contains three browser components that all talk to the same API: the Student Web App (a responsive single-page application), the Organizer Portal used by Faculty and Recruiters, and the Admin Console. The Application Tier is the REST back end running in Docker containers; every request enters through the API Gateway / Controller Layer, which routes it to the Auth and Role Service, Opportunity Service, Application Service, Recommendation Engine or Reporting and Analytics Service. The Application and Opportunity services use the Notification Service to send messages, and all business services reach persistent data only through a single Data Access Layer, which keeps database details in one place and reduces coupling. The Data Tier holds the relational database and the file storage for resumes. Two External Systems are used: the Notification Service calls the Email/SMS Gateway over SMTP or an HTTPS API, and the Recommendation Engine optionally calls an external AI Recommendation API over HTTPS. Dashed arrows are dependencies pointing from the user of a service to the provider.

Component	Responsibility
Student Web App / Organizer Portal / Admin Console	Role-specific browser user interfaces; call the REST API with a JWT.
API Gateway / Controller Layer	Single entry point; validates requests, enforces role-based access and routes to services.
Auth and Role Service	Registration, login, JWT issue/validation, password reset, role checks.
Opportunity Service	Create, approve, publish, search, filter and close opportunities; categories and tags.
Application Service	Submit, track and withdraw applications; applicant lists for organizers.
Recommendation Engine	Matches student profile and survey answers to opportunities; optionally calls an external AI API.
Reporting and Analytics Service	Applicant reports for organizers and the analytics dashboard for Admin.
Notification Service	Deadline reminders and status notifications by e-mail/SMS.
Data Access Layer	Object-relational mapping and file I/O for the database and resume storage.

4. Environmental View

4.1 Deployment Diagram

A deployment diagram shows the mapping of software to hardware: the processing nodes, the artifacts that run on them and the communication paths between nodes. Figure 7 shows how CampusSphere is deployed.

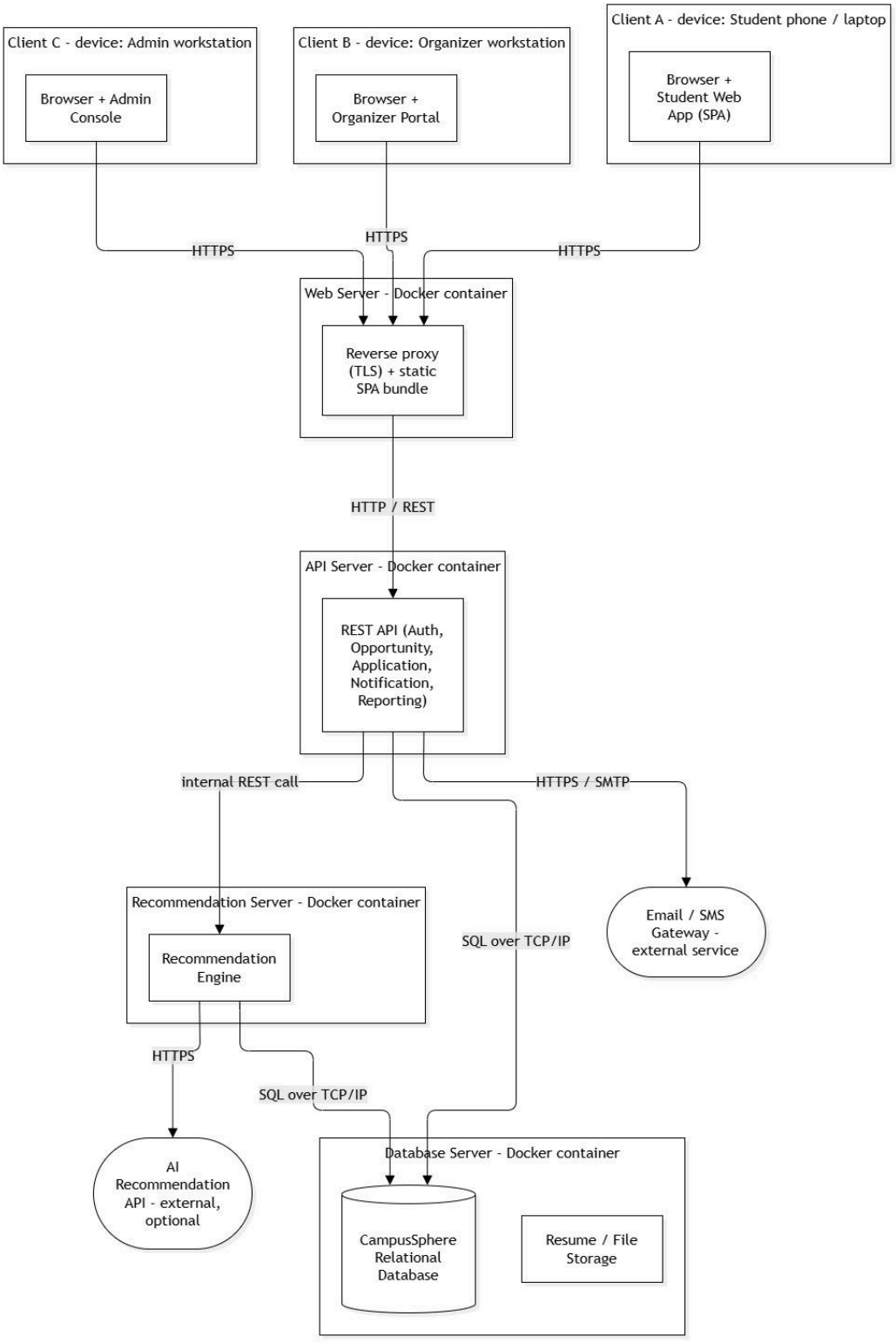


Figure 7: Deployment Diagram - CampusSphere (Environmental View)

Three kinds of client device (the student's phone or laptop, the organizer's workstation and the admin's workstation) each run a browser with the matching web application and connect over HTTPS to the Web Server node, a Docker container that terminates TLS and serves the static single-page application bundle. The Web Server forwards API calls over the internal network to the API Server container, which hosts the REST API with the authentication, opportunity, application, notification and reporting services. The API Server calls the Recommendation Server container, which runs the Recommendation Engine, and both servers read and write the Database Server container over SQL on TCP/IP; this node holds the relational database and the resume file storage. Two external services lie outside the deployment boundary: the API Server reaches the Email/SMS Gateway over HTTPS or SMTP, and the Recommendation Server optionally calls the external AI Recommendation API over HTTPS. As the note on the diagram states, all server-side containers are started together with Docker Compose, which makes the system easy to install on a campus server or a cloud host and allows the recommendation container to be scaled or replaced independently.

Node	Type	Artifacts / role
Client A / B / C	Device	Browser with Student Web App, Organizer Portal or Admin Console
Web Server	Docker container	Reverse proxy with TLS, static SPA bundle
API Server	Docker container	REST API (Auth, Opportunity, Application, Notification, Reporting)
Recommendation Server	Docker container	Recommendation Engine
Database Server	Docker container	Relational database, resume/file storage
Email/SMS Gateway	External service	Delivers e-mail and SMS notifications
AI Recommendation API	External service (optional)	Optional external scoring of recommendations

5. Summary

Taken together, the six diagrams describe how CampusSphere behaves, how it is built and where it runs. The sequence and collaboration diagrams show how objects cooperate in Use Case 3, the two state-charts define the legal life cycles of Opportunity and Application, and the activity diagram shows the end-to-end workflow across student, system and organizer. The component diagram groups the code into tiers and services with a single data access layer, and the deployment diagram maps those components onto containerized nodes and the two external services. All diagrams reuse the class, actor and use-case names of the earlier CampusSphere documents so that every element can be traced back to the Vision and Scope, SRS and Use Cases documents.